

Digital Logic Principles

ECE2300L Final Lab Report

- Overview:
 - Summary of project description
 - top-level block diagram, showing inputs and outputs
 - function table listing all inputs/outputs and a brief description of their functions
 - Methodology:
 - explain how the control path works (state diagram for the FSM finite state machine, design with flip-flops and next-state logic)
 - explain how the datapath works (the logic that moves "data" under the control of the FSM)
 - Simulation with CircuitVerse
 - top, block diagram, logic diagrams of subcircuits
 - wiring diagram(s)
 - in-person or video demonstration of the physical circuit in action
- Shown in class
- timing diagrams for the different scenarios ($\text{curr} < \text{dest}$, $\text{curr} > \text{dest}$, $\text{curr} = \text{dest}$)
- Prototyping

- list of devices / components (expect about 8 devices in addition to the 555 timer and the 7-segment display)
- Conclusion: Issues

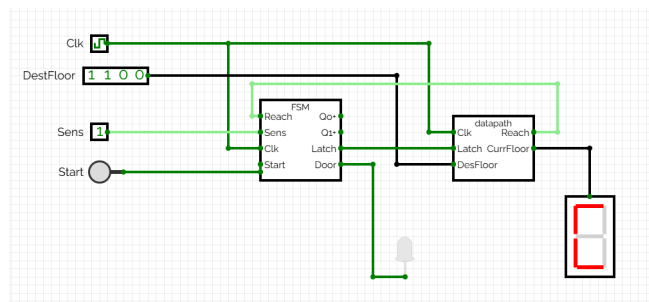
Limitations

Overview

Summary:

The project involves designing a digital elevator controller for a 10-story building (floors 0–9) that allows only one destination request per trip. When the START button is pressed, the selected destination floor (DEST) is stored in a register. The system then compares the current floor (CURR) with the destination using a comparator. If the current floor is lower than the destination, the elevator moves upward. If it is higher, the elevator moves downward, and if both are equal, the elevator remains stationary. Once the elevator reaches the destination floor, the door opens and remains open until a sensor (SENS) detects that no riders are present. After the door closes, the elevator stays at that floor, ready for the next request. The system operates at a clock frequency of 1 Hz, provided by a 555 timer, and uses inputs including the clock, a START push button, a 4-bit destination input, and a sensor. The outputs include a 4-bit current floor display shown on a 7-segment display using an up/down counter, as well as a DOOR signal indicated by an LED or buzzer. Key components include the counter for tracking floor position and a 74LS85 comparator for determining movement direction.

This is the overall top-level block diagram for the circuit, showing the layout of the entire circuit.



Function Table

Start	Reach	Sens	Current State (Q1Q0)	Next State (Q1 ⁺ Q0 ⁺)	Latch	Door
-------	-------	------	----------------------	---	-------	------

0	X	X	WAIT (00)	WAIT (00)	0	0
1	X	X	WAIT (00)	MOVE (01)	1	0
X	0	X	MOVE (01)	MOVE (01)	0	0
X	1	X	MOVE (01)	OPEN (10)	0	0
X	X	0	OPEN (10)	OPEN(10)	0	1
X	X	1	OPEN (10)	WAIT (00)	0	1

The system accepts a START input, a four-bit destination input DEST, and a sensor input SENS, while producing current floor display output and a door control output. When the START button is pressed, the destination value is loaded into a register and held constant for the duration of the trip. The controller continuously compares the current floor value CURR, generated by a four-bit up-down counter, with the stored destination using a magnitude comparator.

Next State Expressions and Output Expressions

$$Q_0^+ = \text{Start } Q_1' Q_0' + \text{Reach}' Q_1' Q_0$$

$$Q_1^+ = \text{Reach } Q_1' Q_0 + \text{Sens}' Q_1 Q_0'$$

$$\text{Latch} = \text{Start } Q_1' Q_0'$$

$$\text{Door} = Q_1 Q_0'$$

After simplification:

$$\text{Latch} = \text{Start } Q_1' Q_0'$$

$$Q_0^+ = \text{Latch} + \text{Reach}' Q_0$$

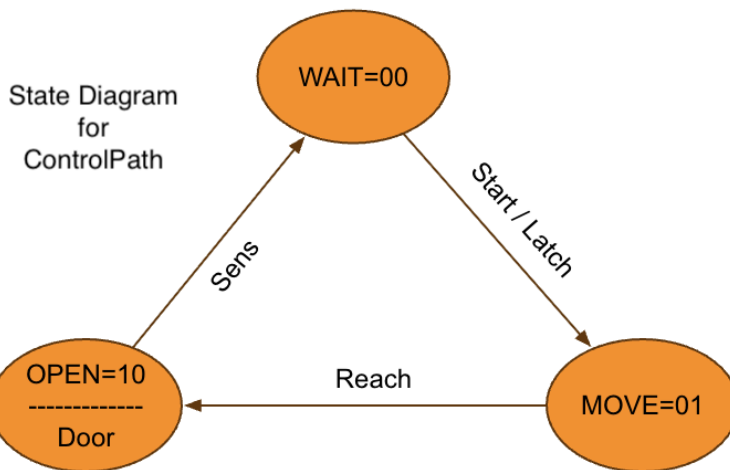
$$Q_1^+ = \text{Reach } Q_0 + \text{Sens}' Q_1$$

$$\text{Door} = Q_1 Q_0'$$

Methodology

The control path of the elevator system is implemented as a finite state machine that coordinates decision-making. The FSM takes in four inputs: Reach, Sens, Start, and Clk. The Reach input is taken from the magnitude comparator, which indicates whether the elevator has arrived at the destination. The Sens input is taken from the photoresistor to sense whether there are any more riders. It produces four outputs: Q1, Q0, Latch, and Door.

State Diagram



The FSM is implemented using two positive-edge-triggered D flip-flops, one for Q1 and one for Q0. The next-state logic is derived from the state transition table by expressing D1 and D0 as Boolean functions of Q1, Q0, and the inputs.

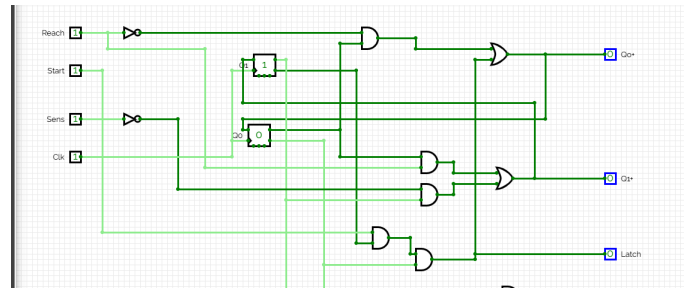
The next state value of Q0 controls entry into and retention of the MOVE state (01). It becomes high when the FSM is in the WAIT state and Start = 1, initiating the transition to MOVE, or when the FSM is already in MOVE and Reach = 0, meaning the elevator has not yet arrived and should continue moving. When Reach transitions to 1, D0 goes low, allowing the FSM to leave the MOVE state and progress to the next state.

The next-state input D1, which determines Q1, controls entry into and retention of the OPEN state (10). It becomes high when the FSM is in the MOVE state and Reach = 1, indicating that the elevator has reached its destination, or when the FSM is already in OPEN and Sens = 0, meaning riders are still present, and the door should remain open. When Sens becomes 1, indicating no riders remain, D1 goes low, along with D0 returning the FSM to the WAIT state (00). The output logic is straightforward because the outputs depend only on the current state. This is an example of a Moore machine behavior.

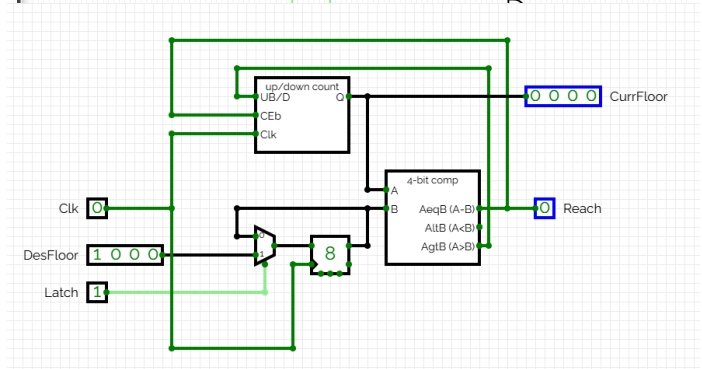
The datapath of the elevator controllers is responsible for storing, comparing, and updating the floor data while operating under the control of the finite state machine. The datapath includes a 4-bit register, a 4-bit up/down counter, and a 74LS85 comparator. When the start button is pressed, the FSM activates Latch to store the selected destination floor to the register. The up/down counter updates the current floor value by counting upward or downward depending on the comparator output. The comparator compares the current floor value with the stored destination and generates Reach when both of the values signal. Once Reach becomes active, the FSM stops the counter and activates the Door output the indicate that the elevator has arrived at the selected floor.

Simulation with CircuitVerse:

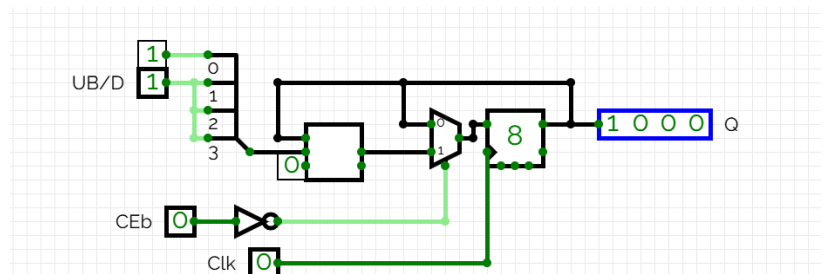
This is the layout of the FSM, which was the first part of the circuit that manages the sequential behavior with the flip-flops.



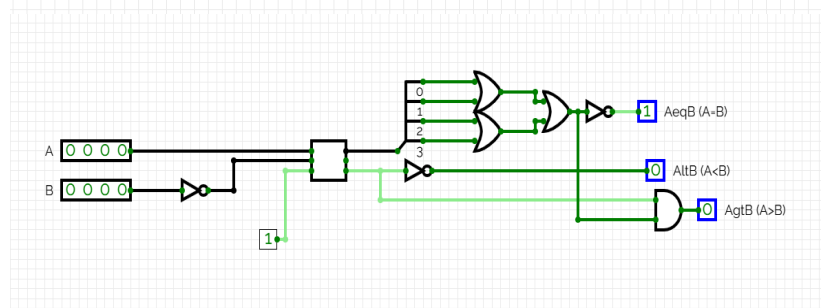
This is the layout of the datapath, which includes the up/down counter, the shift register, and the comparator. This manipulates the data and causes the LED's to light up in the desired order.



This is the up/down counter. This device is in charge of making the elevator go up and down, counting the floors.



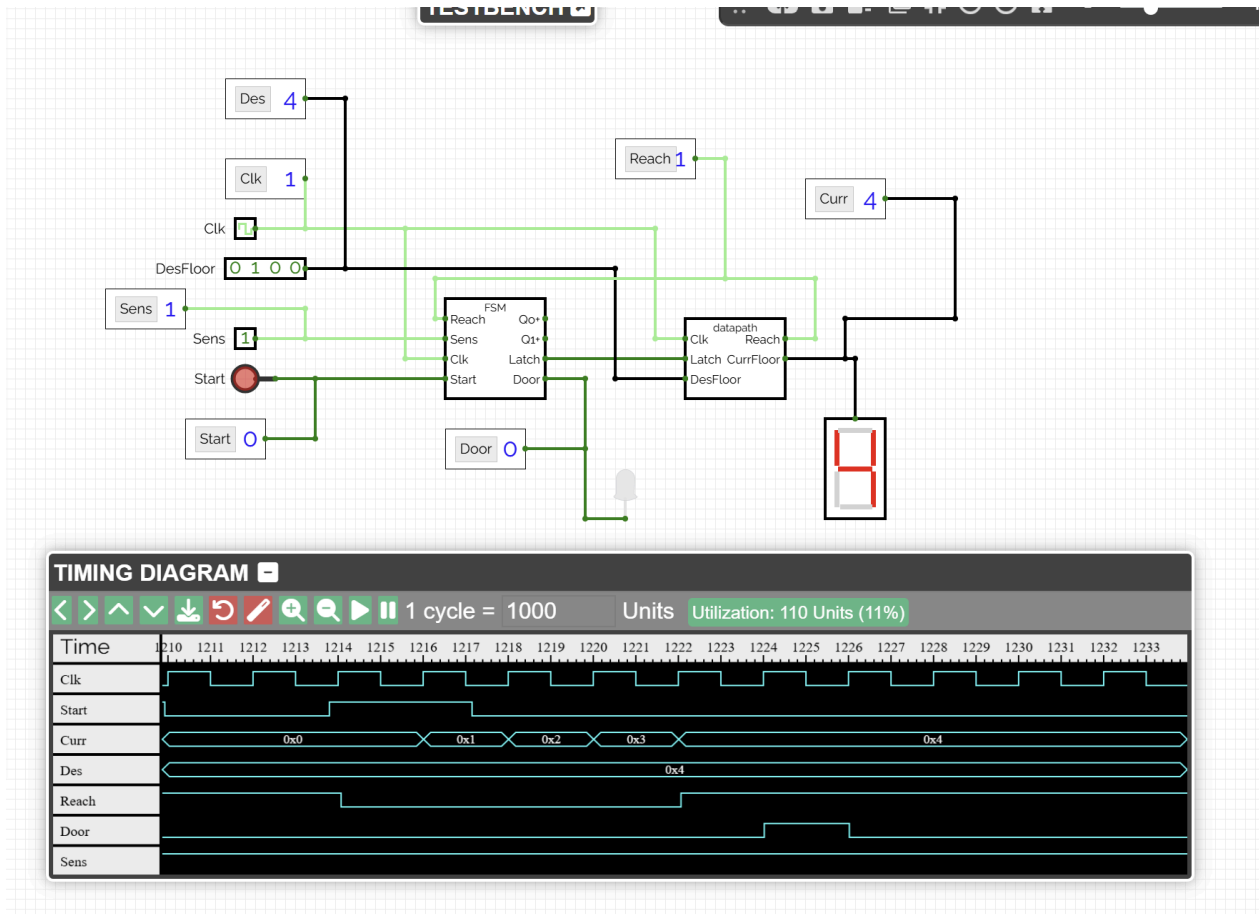
This is the 4-bit comparator. This device allows for reach to be active high when A=B.



Timing Diagrams

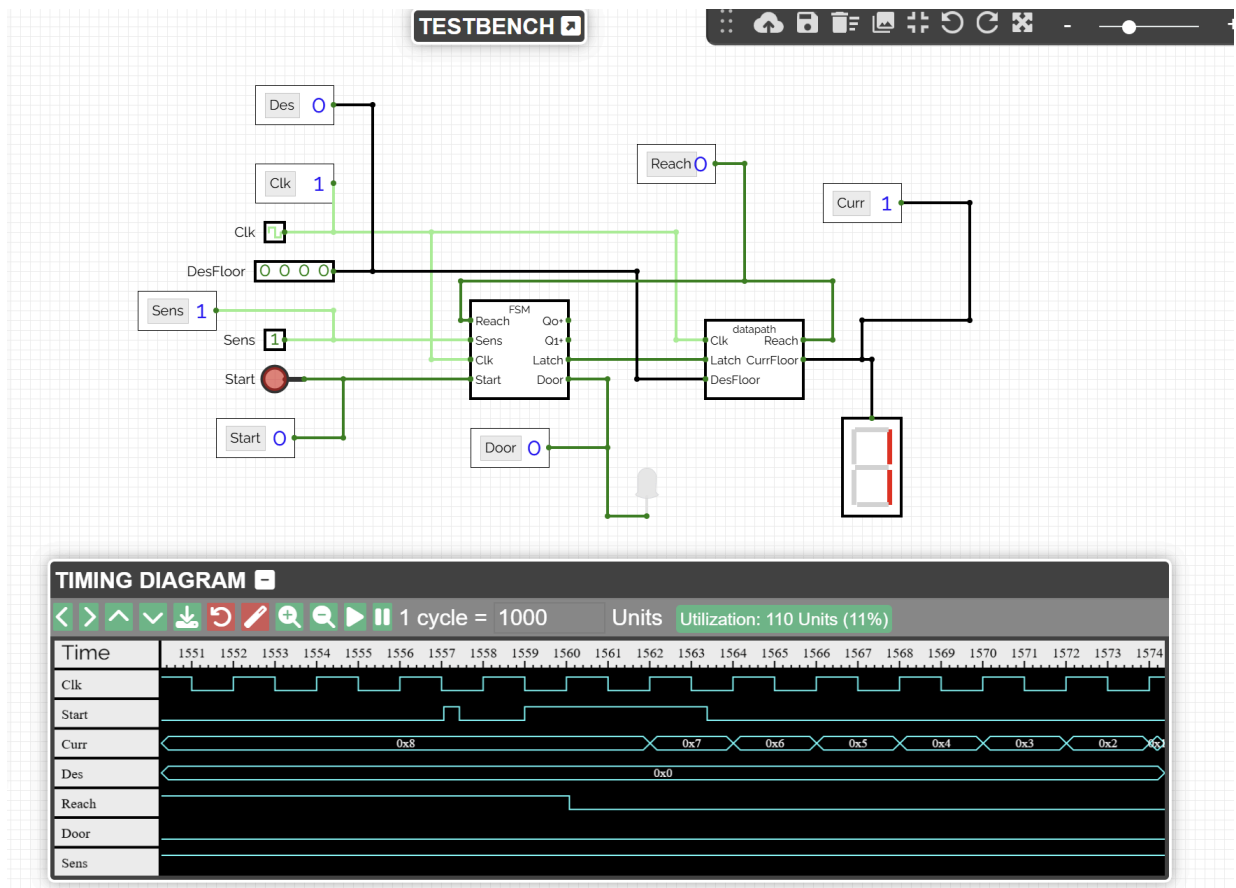
- Curr<Des

In this scenario, the destination is the 4th floor. We started from the ground floor.



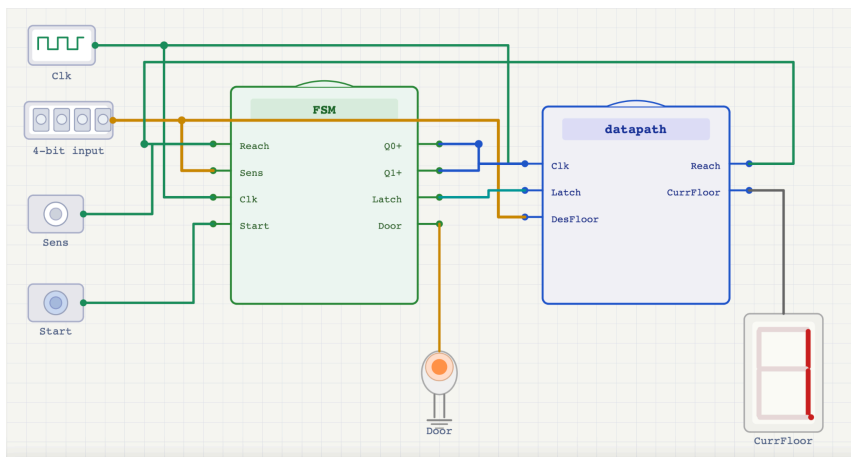
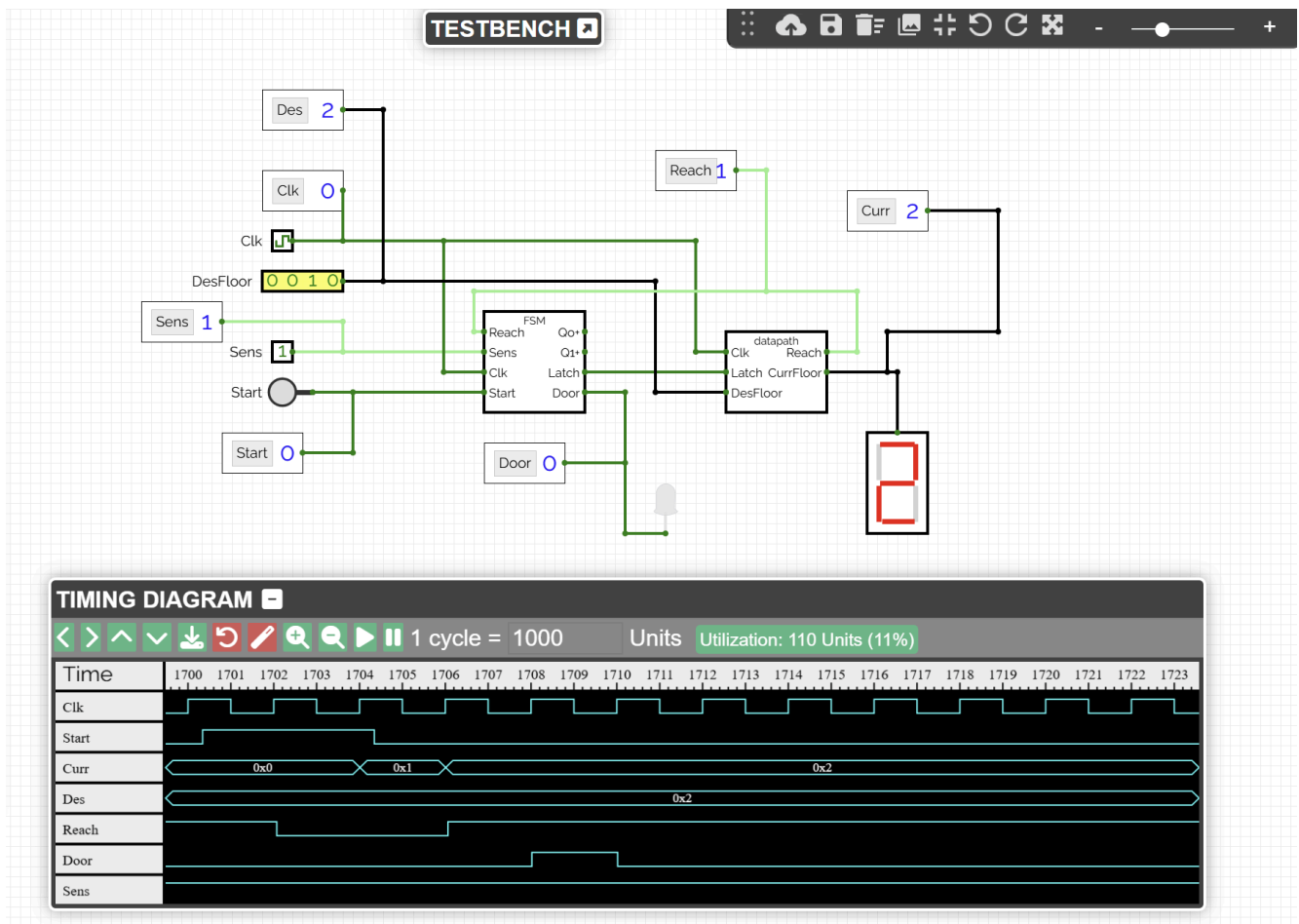
- Curr>Des

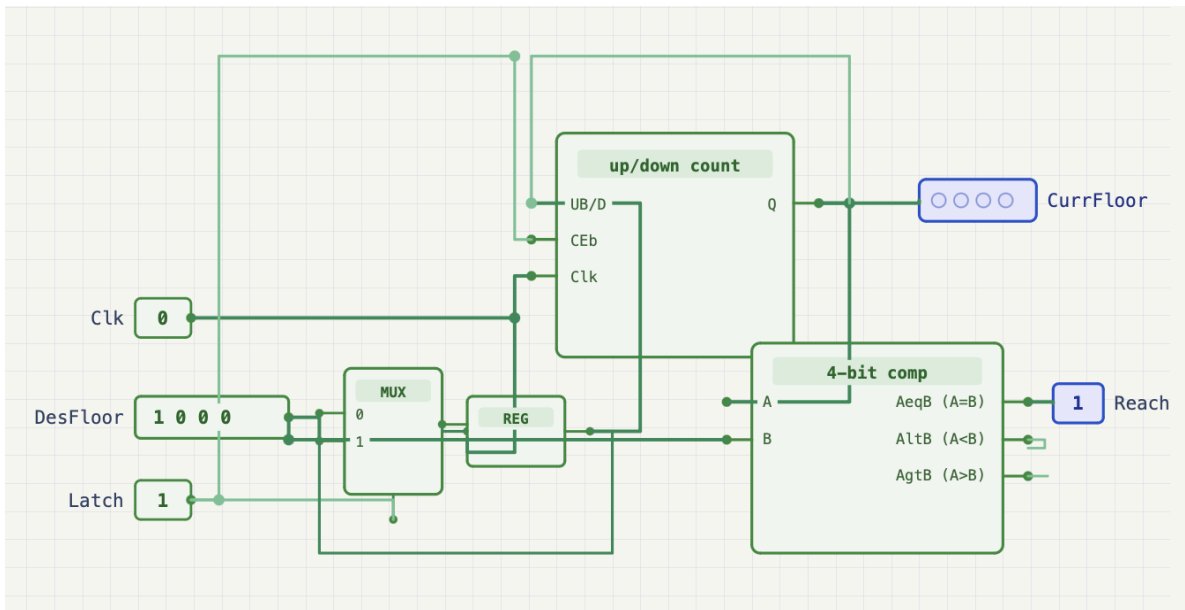
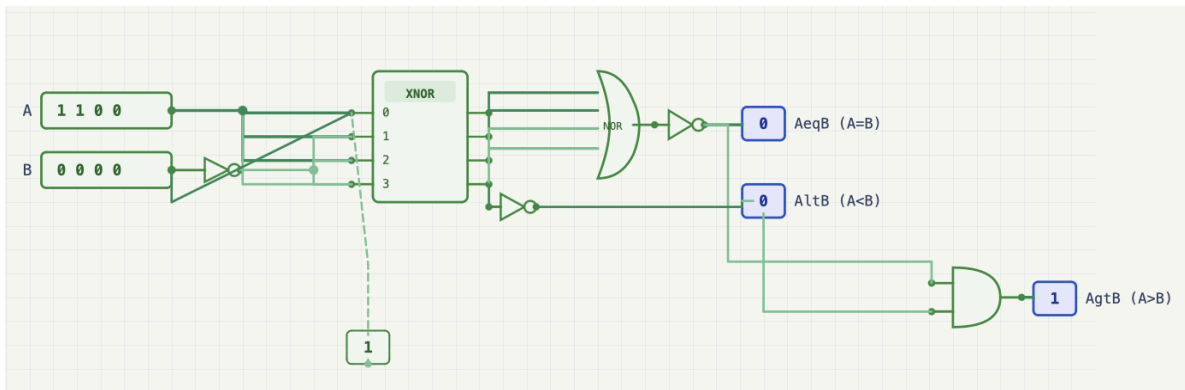
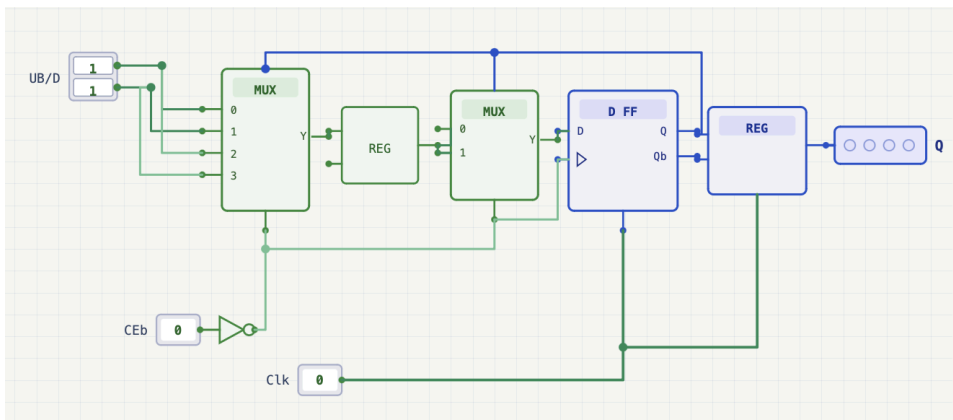
In this scenario, the destination is the ground floor. We started from the 8th floor.



- Curr=Des

We are on the second floor, and the destination is the second floor.





The start of the building process was not too hard; it was just important that we remembered where each input went, so that we did not lose track later. The wiring connection was surprisingly correct from the first draft, not including floating inputs. After changing the latch so that it goes through a NOT gate, we asked the professor for help with our circuit because it was counting weirdly. We were told to ground some floating inputs so that the communication would not be messed up. After correcting this, the counter was working fine; the only issue was that the circuit would start without pressing the start button. After fixing the sensor by using a photoresistor instead of a button, the circuit was working. We were not able to solve the issue with the door light staying on, but we were willing to take partial credit. Overall, it was a fun and interesting project that tied together many lessons learned in the class.

One of the limitations of this elevator controller is the up/down counter. The destination input is 4 bits, which can represent values from 0 to 15, but the building only has floors 0 through 9. Since the counter moves to 15, it results in invalid values on the display. The elevator controller is also extremely simplified. It can only handle one destination input per trip, so it is unable to allow multiple passengers to select different floors or for people from other floors to call the elevator.